M

WeRemember

Complete Project Notebook

ASP.NET Core Razor Pages password vault, Entity Framework database, C# backend concepts, and Chrome extension architecture.

Prepared from the ManPass1 / WeRemember codebase. The goal of this notebook is to explain the theory and the actual project together, so every major file has a clear reason for existing.

1. Big Picture

WeRemember is a local password-manager style application. It has a website where a user can register, log in, and manage saved credentials, and it also has a Chrome extension that can read the current website, suggest matching saved logins, autofill username and password fields, and ask whether to save a new login after a successful sign-in.

The solution folder is still named ManPass1, but the application identity shown to users is WeRemember. This means some internal names, such as the database name ManPass1Db and the encryption purpose string ManPass1.VaultPasswords, remain from the older project name.

Layer	What it does in this project
ASP.NET Core Razor Pages	Builds the website screens such as Home, Register, Login, Vault, Add, View, Edit, and Delete.
C# page models	Contain the server-side logic behind each Razor Page, including validation, database queries, saving, redirects, and authorization checks.
Controllers	Expose JSON API endpoints used by the browser extension. The extension does not submit Razor forms; it calls these API routes.
Entity Framework Core	Maps C# model classes to SQL Server LocalDB tables and lets the app query the database with C# LINQ instead of raw SQL.
Chrome extension	Runs JavaScript in Chrome. It has a popup UI, a background service worker, and a content script injected into websites.
Security services	Uses BCrypt for user password hashing, ASP.NET Core cookies for website sessions, JWT bearer tokens for extension sessions, and Data Protection for vault-password encryption.

2. What .NET And ASP.NET Core Are

.NET is Microsoft's modern development platform for building applications with languages such as C#. It includes the runtime that executes compiled code, the SDK that builds projects, standard libraries, and tools such as the dotnet command.

A framework is a reusable foundation that gives your application a structure and ready-made features. ASP.NET Core is the web framework inside .NET. It handles HTTP requests, routing, authentication, configuration, dependency injection, static files, and server-side rendering.

This project targets net10.0, which means it is built for .NET 10. The project file also uses Microsoft.NET.Sdk.Web, so the compiler and build system know this is a web application, not a console app.

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <AssemblyName>WeRemember</AssemblyName>
    <RootNamespace>WeRemember</RootNamespace>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

TargetFramework selects the .NET version. AssemblyName controls the compiled application name. RootNamespace is the default namespace used by project classes. Nullable enable tells C# to warn about possible null values. ImplicitUsings enable adds common using statements automatically.

3. C# Concepts Used Here

C# is a strongly typed, object-oriented language. Strongly typed means a variable has a known type, such as string, int, bool, User, Credential, or List. Object-oriented means the code is organized around classes that contain data and behavior.

```
public class Credential
{
    public int Id { get; set; }
    public int UserId { get; set; }
    public string WebsiteName { get; set; } = "";
    public string Username { get; set; } = "";
    public string EncryptedPassword { get; set; } = "";
}
```

The syntax public int Id { get; set; } defines a property. A property is like a field with built-in get and set accessors. Entity Framework reads these properties and creates matching database columns.

The = "" default values prevent nullable warnings because the project has Nullable enabled. Without a default value, C# would warn that a non-null string might accidentally be left null.

Syntax	Meaning
namespace WeRemember.Models	Groups related classes and prevents name conflicts.
using Microsoft.EntityFrameworkCore	Imports a library namespace so its classes can be used without fully qualifying every name.
public class User	Declares a class that other parts of the application can use.
private readonly AppDbContext _context	Creates a private dependency field that is assigned once in the constructor.
async Task<IActionResult>	Declares an asynchronous method that eventually returns a web result such as Page, Redirect, Unauthorized, or NotFound.
await _context.SaveChangesAsync()	Pauses this method until the database save finishes without blocking the whole server thread.
var user = ...	Lets the compiler infer the type from the expression on the right side.
new User { Name = Name }	Creates an object and initializes selected properties.

4. Project Structure

A .NET solution is a container that can hold one or more projects. This repository has a solution file named ManPass1.slnx and one main web project folder named ManPass1.

Path	Purpose
Program.cs	Starts and configures the ASP.NET Core application.
ManPass1.csproj	Defines the project target framework and NuGet package dependencies.
Models	Contains C# classes that represent app data, such as User and Credential.
Data/AppDbContext.cs	Defines the Entity Framework database context and tables.
Pages	Contains Razor Pages for the website UI.
Controllers	Contains API controllers for extension login and vault access.
Migrations	Contains database history generated by Entity Framework.
Services	Contains reusable business services, especially vault-password encryption.
wwwroot	Contains public static assets such as CSS, JavaScript, Bootstrap, jQuery, and favicon.
ManPass1Extention	Contains the Chrome extension files. The folder name has a spelling typo, but Chrome can still load it.
appsettings.Development.json	Contains local development settings such as connection string and JWT settings.

5. Program.cs

Program.cs is the startup file. In modern ASP.NET Core, it creates the web app builder, registers services, builds the app, adds middleware, maps endpoints, and starts the server.

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorPages();
builder.Services.AddControllers();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")
    )
);

builder.Services.AddScoped<PasswordEncryptionService>();
```

Services are objects managed by dependency injection. AddRazorPages registers Razor Page support. AddControllers registers API controller support. AddDbContext registers AppDbContext so page models and controllers can ask for it in their constructors. AddScoped means each web request gets its own instance of PasswordEncryptionService.

```
app.UseHttpsRedirection();
app.UseRouting();
app.UseCors("ExtensionPolicy");
app.UseAuthentication();
app.UseAuthorization();
app.MapStaticAssets();
app.MapRazorPages().WithStaticAssets();
app.MapControllers();
app.Run();
```

Middleware order matters. HTTPS redirection runs early. Routing figures out which endpoint matches the URL. CORS is applied before the extension API calls are handled. Authentication identifies the user, authorization checks whether the user is allowed, and endpoint mapping connects Razor Pages and controllers to incoming URLs.

6. Configuration Files

appsettings.json and appsettings.Development.json are JSON configuration files. ASP.NET Core automatically loads them and makes values available through IConfiguration.

```
{
  "ConnectionStrings": {
    "DefaultConnection":
        "Server=(localdb)\\MSSQLLocalDB;Database=ManPass1Db;Trusted_Connection=True;TrustServerCertificate=True;"
  },
  "Jwt": {
    "Key": "WeRemember-Super-Secret-Development-Key-Change-Later-123456789",
    "Issuer": "WeRemember",
    "Audience": "WeRememberExtension"
  }
}
```

The connection string tells Entity Framework where the SQL Server LocalDB database is. The JWT section gives the token issuer, token audience, and signing key used by the extension login flow. In a production system, the JWT key should not be committed in a file; it should come from secure secret storage.

launchSettings.json is used during local development. It defines the local URLs: https://localhost:7189 and http://localhost:5294. The extension points to the HTTPS URL, so the web app must be running on that profile for extension requests to work.

7. Models

A model is a C# class that represents data used by the application. In this project, User represents an account holder and Credential represents one saved login in that user's vault.

```
public class User
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public string Email { get; set; } = "";
    public string PasswordHash { get; set; } = "";
    public List<Credential> Credentials { get; set; } = new();
}
```

User.Id is the primary identifier. Name and Email are account profile values. PasswordHash stores a BCrypt hash, not the original password. Credentials is a navigation property: it tells Entity Framework that one user can have many credential records.

```
public class Credential
{
    public int Id { get; set; }
    public int UserId { get; set; }

    [Required]
    public string WebsiteName { get; set; } = "";

    public string WebsiteUrl { get; set; } = "";

    [Required]
    public string Username { get; set; } = "";

    [Required]
    public string EncryptedPassword { get; set; } = "";

    public User? User { get; set; }
}
```

Credential.UserId is a foreign key back to Users.Id. WebsiteName, Username, and EncryptedPassword are required. WebsiteUrl is optional in the UI, but the current C# type still defaults it to an empty string. User? is another navigation property that lets EF understand the relationship from a credential back to its owner.

8. AppDbContext And Entity Framework

AppDbContext is the bridge between C# objects and the database. It inherits from DbContext, the main Entity Framework Core class for querying and saving data.

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<User> Users { get; set; }
    public DbSet<Credential> Credentials { get; set; }
}
```

DbSet means there is a queryable Users table. DbSet means there is a queryable Credentials table. When code says _context.Users.FirstOrDefaultAsync(...), it is asking EF Core to create and run a SQL query against the Users table.

LINQ is the C# query syntax used here. For example, Where filters records, Select shapes the result, OrderBy sorts records, AnyAsync checks existence, FirstOrDefaultAsync fetches one matching record, and ToListAsync executes the query and returns a list.

```
var credentials = await _context.Credentials
    .Where(c => c.UserId == userId)
    .Select(c => new
    {
        c.Id,
        c.WebsiteName,
        c.WebsiteUrl,
        c.Username
    })
    .ToListAsync();
```

This query only returns credentials belonging to the signed-in user. It also intentionally omits the encrypted password from the list response. The password is only decrypted when a specific credential is requested.

9. Migrations

A migration is a recorded database change. Entity Framework compares the current model classes with the last known database snapshot and generates C# code that can move the database forward or backward.

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    migrationBuilder.CreateTable(
        name: "Users",
        columns: table => new
        {
            Id = table.Column<int>(type: "int", nullable: false)
                .Annotation("SqlServer:Identity", "1, 1"),
            Name = table.Column<string>(type: "nvarchar(max)", nullable: false),
            Email = table.Column<string>(type: "nvarchar(max)", nullable: false),
            PasswordHash = table.Column<string>(type: "nvarchar(max)", nullable: false)
        },
        constraints: table =>
        {
            table.PrimaryKey("PK_Users", x => x.Id);
        });
}
```

Up describes what should happen when the migration is applied. Down describes how to undo it. InitialCreate created the Users table. AddCredentials created the Credentials table and added a foreign key to Users. PersonalDetails created a separate personal details table, and RemovePersonalDetails later removed it.

AppDbContextModelSnapshot is not a normal hand-written source file. It is EF Core's memory of the current model shape. EF uses it to decide what changed when creating the next migration.

10. Razor Pages Theory

Razor Pages is an ASP.NET Core pattern where every page normally has two files. The .cshtml file contains HTML mixed with Razor syntax. The .cshtml.cs file contains a PageModel class with the C# logic for that page.

```
@page
@model WeRemember.Pages.LoginModel

<form method="post">
    <label asp-for="Email" class="form-label"></label>
    <input asp-for="Email" class="form-control" />
    <span asp-validation-for="Email" class="text-danger"></span>
</form>
```

@page makes the file routable as a page. @model connects the view to its C# PageModel. asp-for is a tag helper that binds an HTML element to a C# property. asp-validation-for shows validation messages for that property.

```
[BindProperty]
[Required]
[EmailAddress]
public string Email { get; set; } = "";

public async Task<IActionResult> OnPostAsync()
{
    if (!ModelState.IsValid)
    {
        return Page();
    }
}
```

BindProperty tells Razor Pages to copy form input into the C# property when the form is posted. Required and EmailAddress are validation attributes. ModelState.IsValid checks whether all validation rules passed.

11. Website Page Flow

Page	What happens
Index	Shows the landing page explaining encrypted vault, autofill, and saving new logins.
Register	Collects name, email, and password; validates input; rejects duplicate email; hashes the password; saves the new user.
Login	Checks email and password; creates claims; signs the user in with a cookie; sends the user to the Vault page.
Vault	Requires authorization; reads the current user id from claims; loads only that user's credentials; supports search.
AddCredential	Requires authorization; validates website, username, and password; encrypts the vault password; saves the credential.
ViewCredential	Requires authorization; loads one owned credential; decrypts the password for display.
EditCredential	Requires authorization; loads one owned credential; decrypts it for editing; re-encrypts on save.
DeleteCredential	Requires authorization; confirms the item and deletes only if it belongs to the current user.
Logout	Clears the authentication cookie and redirects back to Login.

The repeated ownership check is important. Pages never fetch a credential by Id alone. They always require both c.Id == id and c.UserId == userId. That prevents one signed-in user from viewing or editing another user's credential by guessing an id.

12. Register And Login

Registration stores a password hash, not the real password. A hash is one-way: the app can verify a future password attempt, but it should not be able to recover the original account password.

```
bool emailExists = await _context.Users
    .AnyAsync(user => user.Email == Email);

string passwordHash = BCrypt.Net.BCrypt.HashPassword(Password);

User newUser = new User
{
    Name = Name,
    Email = Email,
    PasswordHash = passwordHash
};
```

BCrypt is designed for passwords. It is intentionally slow and includes salting internally, which makes large password-guessing attacks harder than they would be with a fast general-purpose hash.

```
bool passwordCorrect =
    BCrypt.Net.BCrypt.Verify(
        Password,
        user.PasswordHash
    );

var claims = new List<Claim>
{
    new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
    new Claim(ClaimTypes.Name, user.Name),
    new Claim(ClaimTypes.Email, user.Email)
};
```

A claim is a fact about the authenticated user. This app stores the user's id, name, and email as claims. Later pages read ClaimTypes.NameIdentifier to know which database records belong to the signed-in user.

13. Cookies And JWT

This project uses two authentication styles. The website uses cookie authentication because a browser can automatically send a secure cookie with each page request. The extension uses JWT bearer authentication because it calls JSON APIs manually with fetch and can add an Authorization header.

```
builder.Services
    .AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Login";
        options.AccessDeniedPath = "/AccessDenied";
        options.Cookie.SameSite = SameSiteMode.None;
        options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
    })
    .AddJwtBearer(JwtBearerDefaults.AuthenticationScheme, options =>
    {
        var jwt = builder.Configuration.GetSection("Jwt");
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = jwt["Issuer"],
            ValidAudience = jwt["Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(jwt["Key"]!))
        };
    });
```

A JWT is a signed token containing claims. The server creates it after extension login. The extension stores it locally and sends it as Authorization: Bearer . The server validates the signature, issuer, audience, and expiration before allowing access.

14. Password Hashing Vs Vault Encryption

The app handles two different kinds of password data. The user's master login password is hashed with BCrypt. Saved website passwords are encrypted with ASP.NET Core Data Protection. Hashing and encryption are not the same.

Concept	Explanation
Hashing	One-way transformation. Used for account passwords because the app only needs to verify a login attempt.
Encryption	Two-way transformation. Used for saved website passwords because the app must later decrypt them for viewing and autofill.
BCrypt	Password hashing algorithm that includes salting and work factor behavior.
Data Protection	ASP.NET Core system for protecting and unprotecting sensitive data using application-managed keys.

```
public class PasswordEncryptionService
{
    private readonly IDataProtector _protector;

    public PasswordEncryptionService(IDataProtectionProvider provider)
    {
        _protector = provider.CreateProtector("ManPass1.VaultPasswords");
    }

    public string Encrypt(string plainText)
    {
        return _protector.Protect(plainText);
    }

    public string Decrypt(string encryptedText)
    {
        return _protector.Unprotect(encryptedText);
    }
}
```

The purpose string ManPass1.VaultPasswords scopes the protector. Data protected for one purpose is not meant to be unprotected with a different purpose. This helps separate different kinds of protected data inside the same app.

15. API Controllers

Controllers are used here for JSON API endpoints. A Razor Page is good for a human-facing webpage; a controller is good for programmatic clients such as the Chrome extension.

```
[ApiController]
[Route("api/auth")]
public class AuthApiController : ControllerBase
{
    [HttpPost("login")]
    public async Task<IActionResult> Login(LoginRequest request)
    {
        // Validate email and password, then return a JWT token.
    }
}
```

ApiController enables API-focused behavior such as request-body binding and validation conventions. Route sets the URL prefix. HttpPost("login") makes the method respond to POST /api/auth/login.

```
[ApiController]
[Route("api/vault")]
[Authorize(AuthenticationSchemes = JwtBearerDefaults.AuthenticationScheme)]
public class VaultApiController : ControllerBase
{
    [HttpGet("credentials")]
    public async Task<IActionResult> GetCredentials() { ... }

    [HttpGet("credential/{id}")]
    public async Task<IActionResult> GetCredential(int id) { ... }

    [HttpPost("credential")]
    public async Task<IActionResult> SaveCredential(SaveCredentialRequest request) { ...
        }
}
```

The vault API explicitly requires JWT bearer authentication. This matters because the website cookie and the extension token are different authentication mechanisms. The extension must supply a valid JWT to read or save vault data.

16. CORS

CORS means Cross-Origin Resource Sharing. Browsers normally restrict a script from one origin when it tries to call another origin. A Chrome extension has a chrome-extension:// origin, while the ASP.NET Core app runs at https://localhost:7189, so the backend must allow extension-origin requests.

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("ExtensionPolicy", policy =>
    {
        policy
            .SetIsOriginAllowed(origin =>
                origin.StartsWith("chrome-extension://"))
            .AllowAnyHeader()
            .AllowAnyMethod()
            .AllowCredentials();
    });
});
```

This policy allows Chrome extension origins, any header, and any HTTP method. The pipeline applies it with app.UseCors("ExtensionPolicy"). Without this, extension fetch calls to the local API may be blocked by the browser.

17. Chrome Extension Overview

A Chrome extension is a small browser app described by manifest.json. This project uses Manifest V3, which is the modern Chrome extension format. The extension has three main pieces: popup.html and popup.js for the toolbar popup, background.js for long-lived extension logic, and content.js for code injected into normal web pages.

File	Role
manifest.json	Declares permissions, background service worker, content script rules, popup page, name, and version.
background.js	Receives messages, calls the ASP.NET Core API, stores the JWT token, and manages pending captured logins.
content.js	Runs inside visited websites, detects login fields, shows suggestions, autofills credentials, captures submitted logins, and displays the save prompt.
popup.html	Defines the extension popup layout and styles.
popup.js	Runs the popup behavior: login to extension, list matching credentials, and trigger autofill.

18. manifest.json

manifest.json is the extension's configuration file. Chrome reads it before running the extension. It tells Chrome what files exist, what permissions are needed, which websites the content script should run on, and what popup should open when the user clicks the extension icon.

```
{
  "manifest_version": 3,
  "name": "WeRemember",
  "version": "1.0",
  "permissions": ["activeTab", "scripting", "storage"],
  "host_permissions": [
    "https://localhost/*",
    "http://*/*",
    "https://*/*"
  ],
  "background": {
    "service_worker": "background.js"
  },
  "content_scripts": [
    {
      "matches": ["http://*/*", "https://*/*"],
      "js": ["content.js"],
      "run_at": "document_idle"
    }
  ],
  "action": {
    "default_popup": "popup.html",
    "default_title": "WeRemember"
  }
}
```

activeTab lets the extension interact with the current tab after user action. scripting allows script-related capabilities. storage lets the extension store the token and temporary login data. host_permissions allow the extension to call localhost and run on normal HTTP and HTTPS websites. run_at document_idle means content.js runs after the page has mostly loaded.

19. background.js

background.js is the extension service worker. It cannot directly see the webpage's DOM like content.js can. Instead, it listens for messages from popup.js or content.js, performs privileged work such as API requests and storage access, then sends responses back.

```
const API_BASE = "https://localhost:7189";

async function getToken() {
    const data = await chrome.storage.local.get("werememberToken");
    return data.werememberToken;
}
```

chrome.storage.local persists data beyond a single tab session. The JWT token is saved there so the user does not have to log into the extension every time the popup opens.

```
chrome.runtime.onMessage.addListener(
    (message, sender, sendResponse) => {
        if (message.type === "LOGIN") {
            fetch(`${API_BASE}/api/auth/login`, {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({
                    email: message.email,
                    password: message.password
                })
            })
            .then(response => response.json())
            .then(async data => {
                await chrome.storage.local.set({
                    werememberToken: data.token
                });
                sendResponse({ success: true });
            });
            return true;
        }
    }
);
```

The return true is important in Chrome extension message listeners. It tells Chrome that sendResponse will be called asynchronously after fetch finishes. Without it, the message channel may close too early.

background.js also stores pending login data in chrome.storage.session using keys like pendingLogin_. Session storage is temporary and tab-scoped behavior is simulated by including the tab id in the key.

20. content.js

content.js runs inside ordinary webpages that match the manifest. It can read and manipulate the page DOM, so it is responsible for finding login inputs, showing suggestions, filling values, detecting submitted logins, and showing the save-password popup.

```
function normalizeHost(host) {
    return host
        .toLowerCase()
        .replace(/^www\./, "");
}

function hostsMatch(savedHost, currentHost) {
    savedHost = normalizeHost(savedHost);
    currentHost = normalizeHost(currentHost);

    return (
        savedHost === currentHost ||
        currentHost.endsWith("." + savedHost) ||
        savedHost.endsWith("." + currentHost)
    );
}
```

Host normalization makes matching friendlier. For example, www.example.com and example.com should be treated as the same site. The subdomain checks also let a credential saved for example.com match login.example.com.

```
function setInputValue(input, value) {
    input.focus();

    const setter =
        Object.getOwnPropertyDescriptor(
            HTMLInputElement.prototype,
            "value"
        )?.set;

    if (setter) {
        setter.call(input, value);
    }
    else {
        input.value = value;
    }

    input.dispatchEvent(new Event("input", { bubbles: true }));
    input.dispatchEvent(new Event("change", { bubbles: true }));
}
```

This is more careful than just setting input.value. Many modern websites use frontend frameworks that listen for input and change events. Dispatching those events helps the website notice the autofilled values.

```
const observer = new MutationObserver(() => {
    attachToInputs();
    tryFillNewPasswordField();
});

observer.observe(
    document.documentElement,
    {
        childList: true,
        subtree: true
    }
);
```

MutationObserver watches for dynamic page changes. Many login pages add inputs after initial load, so the extension keeps checking for newly inserted username and password fields.

21. popup.html And popup.js

popup.html is the small UI shown when the user clicks the extension icon. It contains input fields for logging into WeRemember, a status area, and a container for matching credentials.

popup.js controls the popup. It checks whether a JWT token already exists, asks background.js for credentials, filters them for the current tab's hostname, and sends an AUTOFILL message to content.js when the user clicks Autofill.

```
const tabs = await chrome.tabs.query({
    active: true,
    currentWindow: true
});

const currentUrl = new URL(tab.url);

const result = await chrome.runtime.sendMessage({
    type: "GET_CREDENTIALS"
});
```

chrome.tabs.query finds the active tab. new URL(tab.url) parses the website address. chrome.runtime.sendMessage asks background.js to fetch credential data from the ASP.NET Core backend.

```
await chrome.tabs.sendMessage(
    tab.id,
    {
        type: "AUTOFILL",
        username: credential.username,
        password: credential.password
    }
);
```

tabs.sendMessage sends data into the content script running on the current page. The popup itself cannot fill webpage inputs directly; the content script has to do that work inside the page context.

22. Full Data Flows

Website registration and login

- The user opens Register, fills name, email, and password, then submits the form.
- RegisterModel validates the form, checks whether the email already exists, hashes the password with BCrypt, and saves a User record.
- The user opens Login, submits email and password, and LoginModel verifies the password against PasswordHash.
- If the login is valid, LoginModel creates claims and signs in with cookie authentication.
- The Vault page reads the user id claim and shows only credentials with the same UserId.

Manual credential saving

- The signed-in user opens AddCredential and submits website, URL, username, and password.
- AddCredentialModel validates the fields, reads the signed-in user id, encrypts the website password, and inserts a Credential row.
- The Vault page later displays website and username while masking the password.
- ViewCredential and EditCredential decrypt the saved password only after confirming the credential belongs to the current user.

Extension autofill

- The user logs into the extension popup with the same account email and password.
- popup.js sends LOGIN to background.js, and background.js posts to /api/auth/login.
- AuthApiController verifies the password and returns a signed JWT.
- background.js saves the JWT in chrome.storage.local.
- content.js asks for credentials, filters them by hostname, and shows suggestions near login fields.
- When a credential is selected, content.js asks background.js for the full credential, including decrypted password, then fills the page inputs.

Extension save-new-login prompt

- content.js detects a submitted login form and captures website, username, password, URL, origin, and timestamp.
- background.js stores that temporary login in chrome.storage.session using the tab id.
- After navigation, content.js asks for the pending login and waits to see if the password field disappeared.
- If the form is no longer visible, content.js shows a save prompt.
- When the user clicks Save, background.js posts the captured data to /api/vault/credential with the JWT token.
- VaultApiController encrypts the password and stores it in the database.

23. Important Syntax Cheatsheet

Syntax	Explanation
[Authorize]	Requires the request to come from an authenticated user.
[Authorize(AuthenticationSchemes = JwtBearerDefaults.AuthenticationScheme)]	Requires JWT authentication specifically, which is used by the extension API.
[BindProperty]	Binds posted form values to a Razor Page model property.
[BindProperty(SupportsGet = true)]	Allows a property to be filled from the query string, such as SearchTerm on the Vault page.
[Required]	Makes a field mandatory during model validation.
[EmailAddress]	Checks that a string looks like an email address.
[Url]	Checks that a string looks like a URL.
IActionResult	Represents a web response, such as Page, RedirectToPage, Ok, NotFound, BadRequest, or Unauthorized.
Ok(new { ... })	Returns an HTTP 200 JSON response from an API controller.
Unauthorized()	Returns HTTP 401 when no valid user or token is present.
NotFound()	Returns HTTP 404 when a requested record does not exist or does not belong to the user.
BadRequest(...)	Returns HTTP 400 when the request is invalid.
RedirectToPage("/Vault")	Sends the browser to another Razor Page.
Url.IsLocalUrl(ReturnUrl)	Prevents unsafe redirects to outside websites after login.
ClaimTypes.NameIdentifier	Standard claim type used here to store the user's database id.

24. Security Concepts

Security is central to this project because it stores credentials. The current version is appropriate as a local development project, but a real production password manager would need deeper review, stronger key management, rate limiting, audit logging, secret rotation, hardened deployment, and careful browser-extension review.

Area	How this project handles it
Account password	Stored as BCrypt hash in Users.PasswordHash.
Saved website password	Stored encrypted in Credentials.EncryptedPassword through PasswordEncryptionService.
Website session	Uses ASP.NET Core cookie authentication.

Area	How this project handles it
Extension session	Uses JWT bearer token stored in chrome.storage.local.
Database isolation	Queries filter by UserId, so a user can only access their own vault data.
Extension API access	VaultApiController requires JwtBearer authentication.
CORS	Allows chrome-extension:// origins to call the local backend.
Duplicate capture check	The extension asks /api/vault/credential/exists before showing a save prompt if the same website, username, and password already exist.

One theoretical risk to remember is that extension content scripts interact with arbitrary websites. The code uses escapeWeRememberHtml before placing captured website and username values into the save popup, which helps avoid injecting untrusted text as HTML.

25. Frontend Styling

The website styling lives mainly in wwwroot/css/site.css. It defines the visual identity with CSS custom properties such as --mp-aqua, --mp-blue, --mp-text, and --mp-muted. Bootstrap is also included, so the project uses Bootstrap classes like btn, form-control, navbar, container, mb-3, and text-danger.

```
:root {
  --mp-bg: #eefdfa;
  --mp-text: #102a35;
  --mp-muted: #5d7280;
  --mp-aqua: #02b8a9;
  --mp-blue: #246bfe;
}

.btn-primary {
  border-color: var(--mp-aqua);
  background: var(--mp-aqua);
  color: #ffffff;
}
```

CSS variables make the color system reusable. Razor Pages provide the content and structure, while CSS decides how that content looks. The extension popup repeats the same brand colors inside popup.html and content.js so the extension feels connected to the website.

26. Important Files Summary

File	Important idea
README.md	Explains setup, local database, extension loading, and project structure.
ManPass1.csproj	Declares .NET 10, web SDK, WeRemember assembly name, nullable reference types, and NuGet packages.
Program.cs	Configures Razor Pages, controllers, EF Core, authentication, CORS, middleware, and endpoint mapping.
Models/User.cs	Represents registered users and owns many credentials.
Models/Credential.cs	Represents saved website login details and links back to UserId.
Data/AppDbContext.cs	Makes Users and Credentials available as EF Core tables.
Services/PasswordEncryptionService.cs	Protects and unprotects saved website passwords.
Pages/Register.cshtml(.cs)	Creates user accounts and hashes account passwords.
Pages/Login.cshtml(.cs)	Signs website users in with cookie authentication.

File	Important idea
Pages/Vault.cshtml(.cs)	Lists and searches the signed-in user's saved credentials.
Pages/AddCredential.cshtml(.cs)	Adds a new encrypted credential.
Pages/ViewCredential.cshtml(.cs)	Displays one credential and decrypts its password for viewing.
Pages/EditCredential.cshtml(.cs)	Updates one credential and re-encrypts the password.
Pages/DeleteCredential.cshtml(.cs)	Deletes one credential after ownership validation.
Controllers/AuthApiController.cs	Authenticates extension login and returns a JWT.
Controllers/VaultApiController.cs	Provides JWT-protected credential list, detail, save, and exists endpoints.
Migrations	Shows how EF Core created and evolved SQL tables.
ManPass1Extention/manifest.json	Defines the Chrome extension structure and permissions.
ManPass1Extention/background.js	Acts as extension service worker and API bridge.
ManPass1Extention/content.js	Detects login forms, fills credentials, and prompts to save new logins.
ManPass1Extention/popup.html/js	Shows the extension popup and triggers autofill.

27. Revision Questions

Use these questions to test whether you understand the project instead of only recognizing the filenames.

- Why does the website use cookies while the extension uses JWT tokens?
- Why is the account password hashed but saved website passwords encrypted?
- Why should every credential query include UserId along with credential Id?
- What does DbSet allow the app to do?
- What is the difference between a .cshtml file and a .cshtml.cs file?
- Why does content.js dispatch input and change events after autofill?
- Why does background.js return true from asynchronous message listeners?
- What does a migration's Up method do, and what does Down do?
- Why is CORS needed for the extension to call https://localhost:7189?
- What is the role of appsettings.Development.json during local development?

28. Final Mental Model

Think of WeRemember as two clients sharing one protected backend. The Razor Pages website is the human-facing client for account and vault management. The Chrome extension is the browser-facing client for autofill and capturing new logins. Both rely on the same database and the same security rules.

The database stores users and encrypted credentials. Entity Framework turns C# model classes into SQL tables. Migrations record schema changes. Page models handle website form workflows. Controllers handle extension JSON workflows. Program.cs wires all of those pieces together.

The most important security distinction is simple: the app should verify the user's login password without ever needing to recover it, so that password is hashed. The app must later show and autofill saved website passwords, so those vault passwords are encrypted and decrypted through a dedicated service.